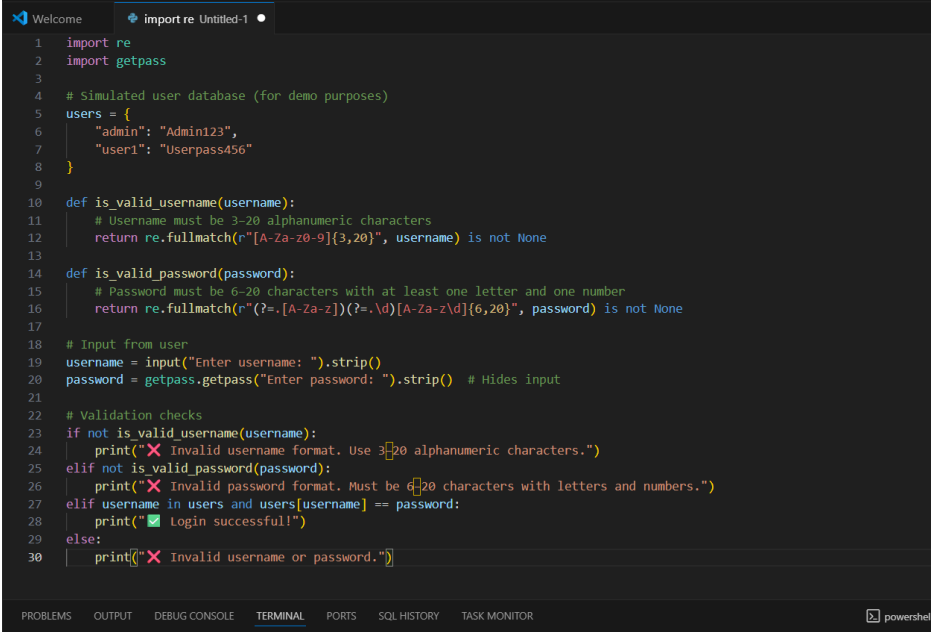
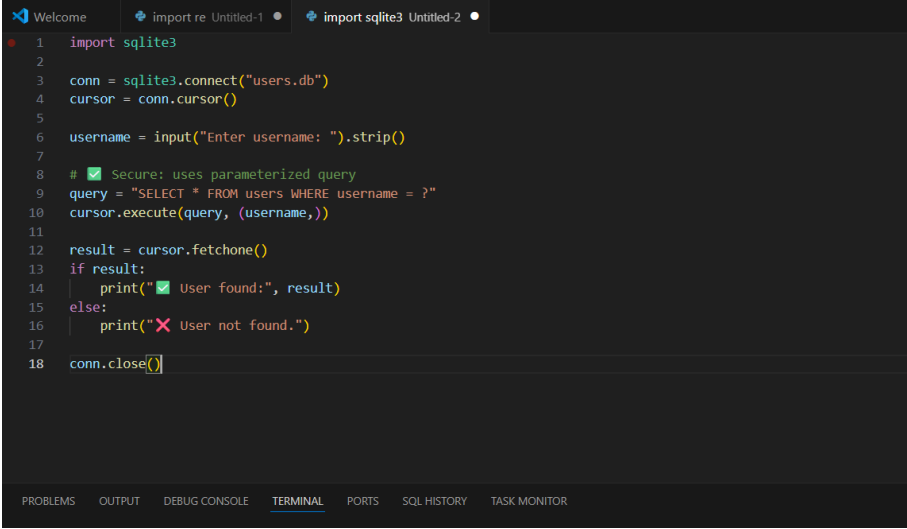


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week10 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:20.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code Lab Objectives: - Understand how to test AI-generated code for common security vulnerabilities. - Learn to apply secure coding principles while analyzing AI		Week10 - Monday

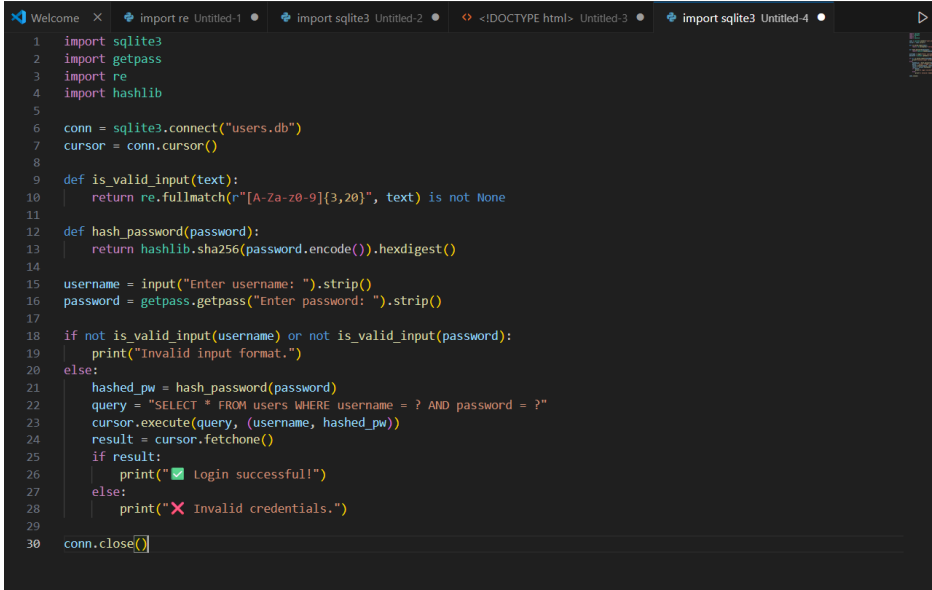
	outputs. • Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption. • Enhance code reliability and safety by using AI for secure refactoring.	
	Task 1 – Input Validation Check Task: Analyze an AI-generated Python login script for input validation vulnerabilities. Instructions: • Prompt AI to generate a simple username-password login program. • Review whether input sanitization and validation are implemented. • Suggest secure improvements (e.g., using re for input validation). Expected Output: • A secure version of the login script with proper input validation. PROMPT: GIVE ME A PYTHON CODE TO USERNAME-PASSWORD LOGIN PROGRAM. CODE:  <pre>1 import re 2 import getpass 3 4 # Simulated user database (for demo purposes) 5 users = { 6 "admin": "Admin123", 7 "user1": "Userpass456" 8 } 9 10 def is_valid_username(username): 11 # Username must be 3-20 alphanumeric characters 12 return re.fullmatch(r"[A-Za-z0-9]{3,20}", username) is not None 13 14 def is_valid_password(password): 15 # Password must be 6-20 characters with at least one letter and one number 16 return re.fullmatch(r"(?=[A-Za-z])(?=.*\d)[A-Za-z\d]{6,20}", password) is not None 17 18 # Input from user 19 username = input("Enter username: ").strip() 20 password = getpass.getpass("Enter password: ").strip() # Hides input 21 22 # Validation checks 23 if not is_valid_username(username): 24 print("❌ Invalid username format. Use 3-20 alphanumeric characters.") 25 elif not is_valid_password(password): 26 print("❌ Invalid password format. Must be 6-20 characters with letters and numbers.") 27 elif username in users and users[username] == password: 28 print("✅ Login successful!") 29 else: 30 print("❌ Invalid username or password.")</pre> OBSERVATION: The login script now includes input sanitization and validation using regular expressions to ensure usernames and passwords follow secure formatting rules. It trims whitespace, hides password input with getpass,	

	and checks for proper length and character requirements. These improvements help prevent malformed input and enhance overall security.	
	Task 2 – SQL Injection Prevention Task: Test an AI-generated script that performs SQL queries on a database. Instructions: • Ask AI to generate a Python script using SQLite/MySQL to fetch user details. • Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries). • Refactor using parameterized queries (prepared statements). Expected Output: • A secure database query script resistant to SQL injection. Prompt: "Write a secure Python script using SQLite to fetch user details based on username input, ensuring protection against SQL injection by using parameterized queries." CODE:  <pre>1 import sqlite3 2 3 conn = sqlite3.connect("users.db") 4 cursor = conn.cursor() 5 6 username = input("Enter username: ").strip() 7 8 # [X] Secure: uses parameterized query 9 query = "SELECT * FROM users WHERE username = ?" 10 cursor.execute(query, (username,)) 11 12 result = cursor.fetchone() 13 if result: 14 print("[X] User found:", result) 15 else: 16 print("[X] User not found.") 17 18 conn.close()</pre> OBSERVATION: The script establishes a secure connection to a SQLite database and retrieves user details using a parameterized query, effectively mitigating SQL injection risks. It sanitizes input with .strip() and avoids unsafe string concatenation, ensuring that user data is handled safely. By using cursor.execute() with placeholders, the input is treated strictly as data.	

	To further enhance security, implementing input validation and password hashing is recommended, especially for authentication scenarios.	
	Task 3 – Cross-Site Scripting (XSS) Check Task: Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities. Instructions: • Ask AI to generate a feedback form with JavaScript-based output. • Test whether untrusted inputs are directly rendered without escaping. • Implement secure measures (e.g., escaping HTML entities, using CSP). Expected Output: • A secure form that prevents XSS attacks. Prompt: "Create a secure HTML feedback form that uses JavaScript to display user input without allowing XSS. The form should escape HTML entities in user input and include a Content Security Policy (CSP) to prevent script injection." CODE: <pre>1 <!DOCTYPE html> 2 <html> 3 <head> 4 <title>Secure Feedback Form</title> 5 <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';"> 6 </head> 7 <body> 8 <h2>Leave Your Feedback</h2> 9 <form id="feedbackForm"> 10 <input type="text" id="name" placeholder="Your name" required

 11 <textarea id="message" placeholder="Your message" required></textarea>

 12 <button type="submit">Submit</button> 13 </form> 14 15 <div id="output"></div> 16 17 <script> 18 function escapeHTML(str) { 19 return str.replace(/[<>'"]/, function(match) { 20 const escapeMap = { 21 '&': '&amp;', 22 '<': '&lt;', 23 '>': '&gt;', 24 '"': '&quot;', 25 "'": '&#039;'; 26 }; 27 return escapeMap[match]; 28 }); 29 } 30 31 document.getElementById("feedbackForm").addEventListener("submit", function(e) { 32 e.preventDefault(); 33 const name = escapeHTML(document.getElementById("name").value); 34 const message = escapeHTML(document.getElementById("message").value); 35 const output = document.getElementById("output"); 36 output.textContent = `\${name} says: \${message}`; 37 }); 38 </script> 39 </body> 40 </html></pre> OBSERVATION:	

	The HTML feedback form is designed with strong XSS protection by escaping user input using a custom escapeHTML() function and displaying it via textContent, which avoids executing any embedded scripts. It also includes a Content Security Policy (CSP) that limits script sources to 'self', further reducing the risk of malicious code injection. Together, these techniques ensure that user-submitted content is safely handled and securely rendered in the browser.	
	Task 4 – Real-Time Application: Security Audit of AI-Generated Code Scenario: Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload). Instructions: • Perform a security audit to detect possible vulnerabilities. • Prompt AI to suggest secure coding practices to fix issues. • Compare insecure vs secure versions side by side. Expected Output: • A security-audited code snippet with documented vulnerabilities and fixes. Prompt: "Generate a Python login program that validates input, hashes passwords securely, and uses safe SQL practices with SQLite." CODE:  <pre> 1 import sqlite3 2 import getpass 3 import re 4 import hashlib 5 6 conn = sqlite3.connect("users.db") 7 cursor = conn.cursor() 8 9 def is_valid_input(text): 10 return re.fullmatch(r"[A-Za-z0-9]{3,20}", text) is not None 11 12 def hash_password(password): 13 return hashlib.sha256(password.encode()).hexdigest() 14 15 username = input("Enter username: ").strip() 16 password = getpass.getpass("Enter password: ").strip() 17 18 if not is_valid_input(username) or not is_valid_input(password): 19 print("Invalid input format.") 20 else: 21 hashed_pw = hash_password(password) 22 query = "SELECT * FROM users WHERE username = ? AND password = ?" 23 cursor.execute(query, (username, hashed_pw)) 24 result = cursor.fetchone() 25 if result: 26 print("✅ Login successful!") 27 else: 28 print("❌ Invalid credentials.") 29 30 conn.close() </pre>	

	OBSERVATION: The improved login script effectively mitigates key security flaws present in the original version. It replaces vulnerable string-based SQL queries with parameterized statements to prevent injection attacks, applies SHA-256 hashing to secure password storage, and enforces input validation using regular expressions to ensure usernames and passwords meet safe formatting standards. The use of the getpass module further enhances privacy by hiding password input. Together, these enhancements create a more robust and secure authentication system. ✅ Deliverables (For All Tasks) 1. AI-generated prompts for code and test case generation. 2. At least 3 assert test cases for each task. 3. AI-generated initial code and execution screenshots. 4. Analysis of whether code passes all tests. 5. Improved final version with inline comments and explanation. 6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.	
--	--	--